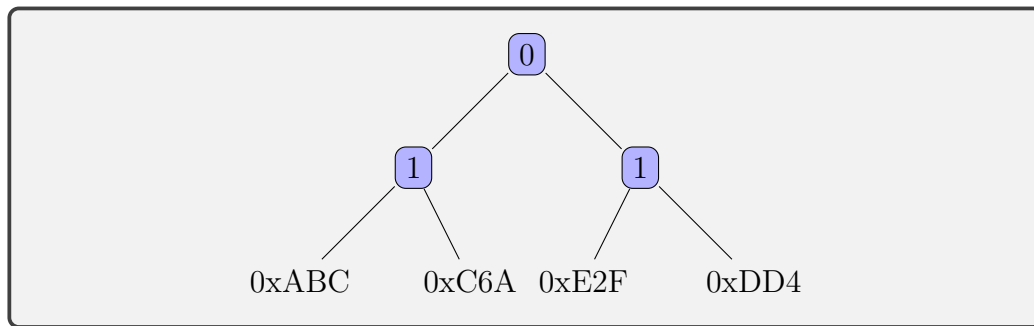


Exercise Sheet

Task 1 – Tree-PLRU

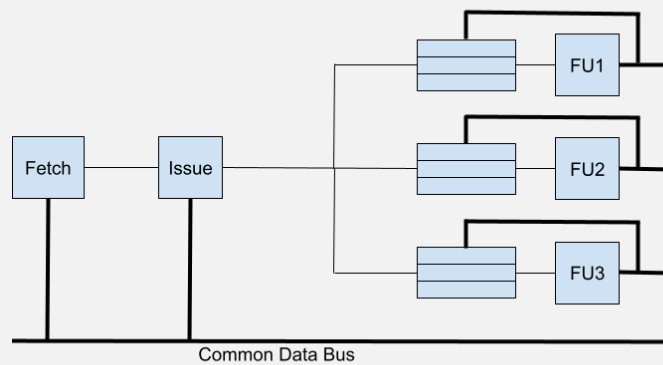
Access sequence: 0xABC, 0xB43, 0xC6A, 0xDD4, 0xABC, 0xE2F

Execute the above sequence in a fully associative cache with 4 cache lines and with Tree-PLRU as replacement strategy. What does the Tree look like after the sequence has finished if at first the cache is empty and all tree nodes point to the left?



Task 2 – Tomasulo's Algorithm

Draw a simplified diagram depicting all necessary components for Tomasulo's algorithm for a CPU with 3 functional units and 3 spaces in each reservation station. In your own words, annotate each step and explain what is necessary for an instruction to enter each step and what to leave it.



Fetch: Instructions are fetched from memory, usually in batches

Issue: Find out which operands are currently available and to which FU the instruction will be issued. Take care of Anti and Output dependencies by using register renaming

Reservation station: Instruction waits until all operands are available. For each operand, it is logged if the operand is already available and who the producer is, as soon as the operand is available, the value is set. The reservation station listens to the common data bus to receive the missing data.

Functional Unit: Executes the instruction and puts the result on the common data bus.

Task 3 – Assembly

Write a RISC-V Assembly program that finds the largest power of 2 less than or equal to a given integer n . For example, if $n = 66$ the result is 64. Structure your program as follows: First, write a value into $t0$, this is your n . Compute your output and also write it into $t0$. The following restrictions apply:

- $n \leq 2^{32-1}$
- $n > 0$
- You may only use instructions from the RV32I Base instruction set and the M-Extension

As a preparation for the exam, write your program on paper. Remember to also document your code by writing comments. Use proper syntax for your documentation, i.e. begin all your comments with a #.

```
li t0,0x7fffffff
li t1,1
loop:
    bltu t0,t1,end
    slli t1,t1,1
    j loop
end:
    srli t0,t1,1
```